

Neuro-Workflow Hackathon Participant Guide

(this guide is also available here: https://github.com/oist/neuro-workflow/blob/main/hackathon/PARTICIPANT_GUIDE.md)

Neuro-Workflow is a node-based model builder for computational neuroscience. This guide is your entry point: it explains the core ideas, lays out the **two hackathon themes**, and links to the detailed file for each step.

What is a node / a workflow?

- **A node is a Python class** that wraps one scientific operation (build a population, add connectivity, run a simulation, compute firing rates). Each node declares its inputs, outputs, and parameters through a schema, so it can be connected to other nodes, even nodes written by other teams or for other simulators.

- **A workflow is a graph of connected nodes.** Running it executes the nodes in order and passes data along the connections.

The same model can be **built and run in two** places:

- ***locally through the NeuroWorkflow Python API** (in a Jupyter notebook), and

- ***visually in the web application (GUI)** on the server.

The two themes

Theme 1: Learn with example workflows:

Run three ready-made point-neuron (NEST/BMTK) notebooks locally to learn the Python API, then reproduce one visually in the GUI.

Theme 2 : Build nodes from your own code:

Bring your own Python; an AI coding agent + the `create-node` skill turns it into nodes and a workflow; you validate locally, upload, and run in the GUI.

Both themes set up with one script that you run from your own working folder and that creates a Python virtual environment for you. **Keep this hackathon kit as read-only reference material, and run the scripts from a separate folder of your own** (examples below).

Before you come

Register: Send your name, affiliation, and email. We create your Neuro-Workflow account in advance.

Laptop: Python 3 + pip. **Theme 1** needs NEST (the setup script installs it via a pip wheel on macOS 15+/Linux x86_64, Python 3.9-3.11; conda fallback otherwise).

If a simulator won't install locally, you can still run on the server, where NEST/TVB are preinstalled.

Bring your code (Theme 2): Any Python you have, a model, preprocessing, analysis, or simulation. Unstructured is fine, that is the starting point.

AI agent (Theme 2): Bring your own Claude Code (preferred) or Codex subscription. If you don't have one, OpenAI API keys are provided, install Codex in that case.

Hackathon pack: UNZIP the zip provided by organizers. Also accessible here (dropbox) →

<https://www.dropbox.com/scl/fi/6lipi9jdos6d4lpqybi3g/hackathon.zip?rlkey=jd6ulrd0ts2aq16r7whvl0vt4&st=k4sz6wvt&dl=0>

Theme 1: Learn NeuroWorkflow with example workflows

You learn NeuroWorkflow using a set of example workflows built with the **NW Model Builder nodes** (population, connectivity, stimulus, simulation, analysis).

Setup (do this before the hackathon)

The three example workflows use the NEST simulator (with BMTK), installed into the same virtual environment as Neuro-Workflow. NEST now ships a PyPI wheel, so the one-command path is simply:

```
# from your own working folder (keep the hackathon kit read-only):  
bash /path/to/hackathon/pointneuron_env_setup/setup_pointneuron.sh
```

(if the above command fails, please check
/path/to/hackathon/pointneuron_env_setup/README.md → ## Path B — conda /
mamba (all platforms))

This creates a venv, installs NeuroWorkflow + NEST + BMTK + JupyterLab, and downloads the three notebooks. It requires CPython 3.9-3.11 on macOS 15+ or Linux x86_64. On other platforms/versions (older macOS, Linux aarch64, Windows), use the conda path instead, both paths are documented in ./pointneuron_env_setup/README.md.

Then launch JupyterLab and run all three notebooks end to end before the hackathon:

```
source .venv/bin/activate      # Windows: .venv\Scripts\activate  
jupyter lab
```

Activity 1: Run the example notebooks locally and learn the Python API

Three example workflows are provided, built with the **NW Model Builder nodes**:

- **NW_SingleCell_PointNeuron.ipynb**, a single point-neuron driven by a current clamp.
- **NW_BalancedNetwork_PointNeuron.ipynb**, an excitatory/inhibitory balanced network.
- **NW_Ring_PointNeuron.ipynb**, a ring-topology network (3 neurons).

Run each notebook locally and follow the NeuroWorkflow Python API as you go. The pattern is the same in all three:

1. Create the nodes.
2. Configure each node's parameters.
3. Create a WorkflowBuilder, add the nodes, and connect their ports.
4. Set the workflow context (e.g. the results path).
5. Build the workflow and execute it.
6. Validate the outputs (confirm the run succeeded and no output is empty / None).

Goal: understand how nodes, parameters, ports, connections, and execution fit together, the foundation for everything that follows.

Activity 2: Reproduce one workflow in the GUI on the server

Choose any one of the three workflows and rebuild it visually in the web application on the server. The NW Model Builder nodes are already available on the server, so you do not create any node here, you assemble the workflow:

1. Add the required nodes from the palette.
2. Connect their ports to form the workflow graph.
3. Configure the parameters of each node.
4. Build the workflow, generate the code, and run it.

Do this with the help of the in-app AI assistant, which can add nodes, connect them, and set parameters for you on request.

Goal: see that the same model you ran locally in Python can be reproduced and executed visually on the server.

Theme 2: Build nodes from your own code with AI agents

This theme is similar to Theme 1, except that here you create the nodes and the workflow from your own source code, with the help of an AI coding agent:

- You bring your own Python code (a model, a preprocessing or analysis script, a simulation, unstructured is fine; that is the starting point).
- You use an **AI coding agent** (Claude Code or Codex) guided by our **create-node skill** to turn that code into Neuro-Workflow nodes and a workflow. The skill encodes the conventions and checks we developed building and testing nodes by hand, so the agent produces clean, uploadable nodes.
- You run the generated nodes and workflow **locally** and validate the outputs.
- You **upload the nodes** to the server through the GUI.
- You assemble the workflow on the server with the in-app **AI assistant**, the same way as in Theme 1.

Setup

From your own working folder, run the one-command script for your agent (keep this kit read-only):

Claude Code:

```
bash /path/to/hackathon/quick_setup/setup_claude.sh
```

Codex:

```
bash /path/to/hackathon/quick_setup/setup_codex.sh
```

Each script creates a venv, installs a pinned NeuroWorkflow + JupyterLab + matplotlib, makes a **source_code/** folder (drop your code here) and a **my_nodes/** folder (the agent writes here), and installs the **create-node skill for your agent**. Full details, both agents, the manual fallback, and a dependency-free ****green-run**** sanity check , are in **./SETUP.md**

Start Claude Code or Codex in this folder and prompt the agent:

For example:

Claude Code:

Use the /create-node skill. Look in the source_code/ folder, find the main Python file(s), analyze the code, and propose the nodes to build. Ask me to confirm the breakdown before you write anything. Save the nodes AND two workflow files (.py and .ipynb) in the my_nodes/ folder. Then smoke-test each node and run the workflow so we can see the outputs.

Codex:

Use the create-node skill (\$create-node). Look in the source_code/ folder, find the main Python file(s), analyze the code, and propose the nodes to build. Ask me to confirm the breakdown before you write anything. Save the nodes AND two workflow files (.py and .ipynb) in the my_nodes/ folder. Then smoke-test each node and run the workflow so we can see the outputs.

More details here→ <https://github.com/oist/neuro-workflow/blob/main/hackathon/SETUP.md>

Step-by-step walkthrough

The complete four-step agenda, build nodes locally, test them with `check_node.py`, upload them, and run the workflow on the server, is in `./README.md`. Start there once your environment is set up. If you didn't bring your own code, ask the organizers.

Running on the server

For this hackathon, workflows run directly on the application server, so please keep example runs lightweight (short simulations, modest network sizes). A dedicated HPC compute backend is not used in this round.

Support

A built-in AI assistant is available inside the platform for participants who prefer not to use Claude Code or Codex directly. Working locally with a coding agent is the recommended path for Theme 2, it gives you full control over node development. If a simulator install fails or anything blocks you, ask an organizer.